

```

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<ll, ll> pll;
typedef vector<ll> vll;

#define forn(i, s, e) for (ll i = (s); i < (e); i++)
#define sz(u) ((ll) u.size())
#define ln "\n"

#ifdef DEBUG
#define trace(u) u
#define _
#else
#define trace(u)
#define _ ios::sync_with_stdio(0); cin.tie(0)
#endif

#define debug(u) trace(cout << #u " = " << u << ln)
#define debugv(v) trace(cout << #v ": "; for (auto xx : v) cout << xx << " "; cout << ln)

const ll INF = 0x3f3f3f3f3f3f3f3f;

// Segment Tree (Range Query + Range Update)
//
// Tree for range queries with a customizable combine; supports range updates and range queries.
//
// complexity: O(log N) per op, O(N) to build

struct Node {
    ll sum, cnt = 1;
    Node operator*(const Node &o) const { return {sum + o.sum, cnt + o.cnt}; }
};

struct Update {
    ll add = 0;

    Node operator()(const Node &n) const {
        return {n.sum + add * n.cnt, n.cnt};
    }

    Update operator+(const Update &o) const {
        Update res = *this;
        res.add += o.add;
        return res;
    }
};

template<typename T, typename U> struct segtree {
    ll s, h;

    T id;
    vector<T> val;

    U noop;
    vector<bool> dirty;
    vector<U> prop;

    segtree(ll ts, T tid = T(), U tnoop = U()) {
        id = tid, noop = tnoop;
        for (s = 1, h = 1; s < ts; ) s *= 2, h++;

        val.assign(2*s, id);
        dirty.assign(2*s, false);
        prop.assign(2*s, noop);
    }

    void set_leaves(vector<T> &lvs) {
        copy(lvs.begin(), lvs.end(), val.begin() + s);

        for (ll i = s - 1; i > 0; i--) val[i] = val[2 * i] * val[2 * i + 1];
        dirty.assign(2*s, false);
        prop.assign(2*s, noop);
    }

    void apply(ll i, U &upd) {
        val[i] = upd(val[i]);
        if (i < s) {
            prop[i] = prop[i] + upd;
            dirty[i] = true;
        }
    }
};

```

```
void pull(ll i) {
    for (ll l = i/2; l; l /= 2) {
        T comb = val[2*l] * val[2*l+1];
        val[l] = prop[l](comb);
    }
}

void push(ll i) {
    for (ll th = h; th > 0; th--) {
        ll l = i >> th;

        if (dirty[l]) {
            apply(2*l, prop[l]);
            apply(2*l+1, prop[l]);

            prop[l] = noop;
            dirty[l] = false;
        }
    }
}

void update(ll i, ll j, U upd) {
    i += s, j += s;
    push(i), push(j);

    for (ll l = i, r = j; l <= r; l /= 2, r /= 2) {
        if ((l&1) == 1) apply(l++, upd);
        if ((r&1) == 0) apply(r--, upd);
    }

    pull(i), pull(j);
}

T query(ll i, ll j){
    i += s, j += s;
    push(i), push(j);

    T rl = id, rr = id;
    for(; i <= j; i /= 2, j /= 2){
        if((i&1) == 1) rl = rl * val[i++];
        if((j&1) == 0) rr = val[j--] * rr;
    }
    return rl * rr;
}

int main() {
    _;
    ll n, q; cin >> n >> q;
    vector<Node> vec(n);
    forn(i,0,n) cin >> vec[i].sum;

    segtree<Node, Update> seg(n);
    seg.set_leaves(vec);

    while (q--){
        ll t;
        cin >> t;
        if(t == 1){
            ll a, b, u; cin >> a >> b >> u;
            a--, b--;
            seg.update(a,b,{u});
        }else{
            ll k; cin >> k;
            k--;
            cout << seg.query(k,k).sum << ln;
        }
    }

    return 0;
}
```

```

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<ll, ll> pll;
typedef vector<ll> vll;

#define forn(i, s, e) for (ll i = (s); i < (e); i++)
#define sz(u) ((ll) u.size())
#define ln "\n"

#ifdef DEBUG
#define trace(u) u
#define _
#else
#define trace(u)
#define _ ios::sync_with_stdio(0); cin.tie(0)
#endif

#define debug(u) trace(cout << #u " = " << u << ln)
#define debugv(v) trace(cout << #v ": "; for (auto xx : v) cout << xx << " "; cout << ln)

const ll INF = 0x3f3f3f3f3f3f3f3f;

// Dinic's Maximum Flow (with Scaling)
//
// Computes max flow using Dinic's algorithm with optional capacity scaling to speed up BFS levels.
//
// complexity: O(E V^2) worst-case, O(E)

struct dinitz {
    const bool scaling;
    ll lim;
    struct edge {
        ll to, cap, rev, flow;
        bool res;
        edge(ll to_, ll cap_, ll rev_, bool res_)
            : to(to_), cap(cap_), rev(rev_), flow(0), res(res_) {}
    };

    vector<vector<edge>> g;
    vll lev, beg;
    ll F;
    dinitz(ll n, bool scaling_) : scaling(scaling_), g(n), F(0){}

    void add(ll a, ll b, ll c) {
        g[a].emplace_back(b, c, sz(g[b]), false);
        g[b].emplace_back(a, 0, sz(g[a])-1, true);
    }

    bool bfs(ll s, ll t) {
        lev = vll(sz(g), -1); lev[s] = 0;
        beg = vll(sz(g), 0);
        queue<ll> q; q.push(s);
        while (sz(q)) {
            ll u = q.front(); q.pop();
            for (auto& i : g[u]) {
                if (lev[i.to] != -1 || (i.flow == i.cap)) continue;
                if (scaling && i.cap - i.flow < lim) continue;
                lev[i.to] = lev[u] + 1;
                q.push(i.to);
            }
        }
        return lev[t] != -1;
    }

    ll dfs(ll v, ll t, ll f = INF) {
        if (!f || v == t) return f;
        for (ll& i = beg[v]; i < sz(g[v]); i++) {
            auto& e = g[v][i];
            if (lev[e.to] != lev[v] + 1) continue;
            ll foi = dfs(e.to, t, min(f, e.cap - e.flow));
            if (!foi) continue;
            e.flow += foi, g[e.to][e.rev].flow -= foi;
            return foi;
        }
        return 0;
    }

    ll max_flow(ll s, ll t) {
        for (lim = scaling ? (1ll<<30) : 1; lim; lim /= 2)
            while (bfs(s, t)) while (ll ff = dfs(s, t)) F += ff;
    }
};

```

```
    return F;
}

vector<pll> min_cut() {
    vector<pll> cut;
    forn(u, 0, sz(g))
        if (lev[u] != -1)
            for (auto& e : g[u])
                if (lev[e.to] == -1 && !e.res)
                    cut.push_back({u, e.to});
    return cut;
}

void reset() {
    F = 0;
    for (auto& edges : g) for (auto& e : edges) e.flow = 0;
}

};

int main() {
    _;
    ll n, m; cin >> n >> m;

    dinitz fluxo(n, true);

    forn(i, 0, m) {
        ll a, b;
        cin >> a >> b;
        a--, b--;
        fluxo.add(a, b, 1);
    }

    ll k = fluxo.max_flow(0, n-1);
    vector<vll> g(n);
    auto gf = fluxo.g;
    forn(i, 0, n) {
        for(auto [to, cap, rev, flow, res] : gf[i]) {
            if(res) continue;
            if(flow == 0) continue;
            g[i].push_back(to);
        }
    }

    cout << k << ln;
    vll resp;
    for(ll v: g[0]) {
        resp.push_back(0);
        resp.push_back(v);
        while(v != n-1) {

            ll nv = g[v].back();
            g[v].pop_back();
            v = nv;
            resp.push_back(v);
        }
        cout << sz(resp) << ln;
        forn(i, 0, sz(resp)) cout << resp[i]+1 << " \n"[i == sz(resp)-1];
        resp.clear();
    }

    return 0;
}
```

```

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<ll, ll> pll;
typedef vector<ll> vll;

#define forn(i, s, e) for(ll i = (s); i < (e); i++)
#define sz(x) ((ll) x.size())
#define ln "\n"

#ifdef DEBUG
#define trace(x) x
#define _ (void)0
#else
#define trace(x) (void)0
#define _ ios_base::sync_with_stdio(false), cin.tie(NULL)
#endif

#define debugv(v) trace({cout << #v": "; for (auto x : v) cout<< x << " "; cout << ln;})
#define debug(x) trace(cout << __LINE__ << ": " #x " = " << x << ln)

const ll INF = 0x3f3f3f3f3f3f3f3f;

// Hungarian algorithm â200\224 min-cost perfect matching
// a[i][j] = cost of assigning worker i to job j (n workers, m >= n jobs)
// Returns {min cost, assignment}: assignment[i] = j
pair<ll, vll> hungarian(const vector<vll> &a) {
    if (a.empty()) return {0, {}};
    ll n = sz(a) + 1, m = sz(a[0]) + 1;
    vll u(n), v(m), p(m), ans(n - 1);
    forn(i, 1, n) {
        p[0] = i;
        ll j0 = 0; // "dummy" worker 0
        vll dist(m, INF), pre(m, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            ll i0 = p[j0], j1 = 0, delta = INF;
            forn(j, 1, m) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            forn(j, 0, m) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            ll j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
        forn(j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans}; // min cost
    }
}

int main() {
    _;
    ll n; cin >> n;
    vector<vll> a(n, vll(n));

    forn(i, 0, n)
        forn(j, 0, n)
            cin >> a[i][j];

    auto [val, ans] = hungarian(a);

    cout << val << ln;
    forn(i, 0, n) {
        cout << i+1 << " " << ans[i]+1 << ln;
    }
    return 0;
}

```